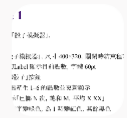
物件導向程式設計II

姓名:李欣曈 學號:A1143302

HW2-0918-AI 對話紀錄



這是要用Java寫的程式，請問最一開始需要設立哪些元件和變數

GUI 元件 (介面控制項)

- **JFrame (主視窗)**：作為程式基底，需設定標題為「骰子模擬器」、尺寸 400x320，並設定 `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` (關閉時結束程式) 以及 `setLocationRelativeTo(null)` (開啟時置中)。 PNG
- **JLabel (統計資訊標籤)**：放置於視窗上方，用於動態更新並顯示「已擲 N 次, 總和 M, 平均 X.XX」。 PNG
- **JLabel (點數顯示標籤)**：放置於視窗中央，需設定字體大小為 60pt。此元件後續需根據點數改變文字顏色（6 為綠色，1 為紅色，其餘黑色）。 PNG
- **JButton (擲骰子按鈕)**：放置於視窗下方，文字設定為「擲骰子」，並需綁定點擊事件監聽器 (`ActionListener`) 來觸發隨機數產生。 PNG

資料變數 (狀態紀錄)

- **int rollCount (擲骰次數)**：整數變數，對應題目中的「N」，初始值設為 0，每次按下按鈕時加 1。 PNG
- **int totalSum (點數總和)**：整數變數，對應題目中的「M」，初始值設為 0，每次按下按鈕時累加當次擲出的點數。 PNG
- **Random 物件 (隨機數產生器)**：宣告 `java.util.Random` 用於產生 1~6 的隨機整數。若不宣告此物件，也可在事件內直接使用 `(int)(Math.random() * 6) + 1` 來實作。 PNG

(註：平均值 X.XX 不一定要設為全域變數，可在按鈕點擊事件中直接以 `(double) totalSum / rollCount` 計算，並用 `String.format` 格式化至小數點後兩位即可。) PNG

設定視窗標題要怎麼在開啟時置中

在 Java Swing 中，要設定視窗標題以及讓視窗在開啟時自動置於螢幕中央，您可以使用 `JFrame` 提供的內建方法。

要讓視窗置中的關鍵程式碼是 `setLocationRelativeTo(null);`。請注意，這行程式碼通常需要寫在設定好視窗大小 (`setSize`) 之後，以及顯示視窗 (`setVisible`) 之前，才能正確發揮作用。

以下是結合您題目規格的基礎視窗設定範例：

```java
import javax.swing.JFrame;

public class DiceSimulator {
    public static void main(String[] args) {
        // 1. 建立 JFrame 物件，並在括號內直接設定「視窗標題」
        JFrame frame = new JFrame("骰子模擬器");

        // (或是使用 frame.setTitle("骰子模擬器"); 來設定)

        // 2. 設定視窗尺寸為 400x320
        frame.setSize(400, 320);

        // 3. 設定當關閉視窗時，自動結束程式
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // 4. 設定視窗開啟時「置中」 (傳入 null 代表不依附其他元件，預設為螢幕正中央)
        frame.setLocationRelativeTo(null);

        // 5. 最後設定視窗為可見，讓它顯示出來
        frame.setVisible(true);
    }
}
```

重點步驟總結：

1. 使用 `new JFrame("標題名稱")` 來設定標題。
2. 使用 `setSize(寬, 高)` 給定明確尺寸。
3. 呼叫 `setLocationRelativeTo(null)` 讓視窗對齊螢幕中央。

frame.setVisible(true);應該要擺在其他元件也建立好之後吧?

沒錯，您的觀念非常正確！

`frame.setVisible(true);` 確實應該要放在**所有元件都建立、設定完畢，並且加入到視窗（或容器）之後的最後一步**。

為什麼要放在最後？
如果提早將視窗設定為可見（ `setVisible(true)` ），程式就會立刻把視窗畫在螢幕上。這時候如果視窗裡還是空的，使用者就會看到一個空白視窗。接著程式繼續執行把按鈕、標籤等元件加進去，但因為視窗已經畫好了，這些後加進去的元件**可能會顯示不出來**（畫面沒有即時更新），通常要等到使用者去拖曳、縮放視窗大小，觸發了畫面的「重繪 (Repaint)」，元件才會突然跑出來。

因此，標準的 Swing 視窗建立流程通常會是這樣的順序：

Java

```java
import javax.swing.*;
import java.awt.*;

public class DiceSimulator {
    public static void main(String[] args) {
        // 1. 建立主視窗與基本設定
        JFrame frame = new JFrame("骰子模擬器");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 320);

        // --- 這裡開始建立並加入其他元件 ---

        // (選擇性) 設定排版管理員，例如 BorderLayout
        frame.setLayout(new BorderLayout());

        // 2. 建立元件
        JLabel statsLabel = new JLabel("已擲 0 次, 總和 0, 平均 0.00", SwingConstants.C
        JLabel diceLabel = new JLabel("?", SwingConstants.CENTER);
        diceLabel.setFont(new Font("微軟正黑體", Font.BOLD, 60)); // 設定 60pt 字體
        JButton rollButton = new JButton("擲骰子");

        // 3. 將元件加入到視窗中 (依照題目要求的上、中、下位置)
        frame.add(statsLabel, BorderLayout.NORTH);
        frame.add(diceLabel, BorderLayout.CENTER);
        frame.add(rollButton, BorderLayout.SOUTH);

        // --- 元件都加完了，最後才顯示視窗 ---

        // 4. 設定視窗置中 (記得放在 setSize 之後)
        frame.setLocationRelativeTo(null);

        // 5. 將視窗設定為可見 (這一行一定要放在最後面！)
```

Java

```java
        // 5. 將視窗設定為可見 (這一行一定要放在最後面！)
        frame.setVisible(true);
    }
}
```

把 `setVisible(true)` 留到壓軸，就能確保視窗彈出來的瞬間，所有辛苦刻好的按鈕和文字都已經整整齊齊地排在畫面上囉！

上方顯示的文字和擲骰子按鈕不用設定字體跟大小嗎?以及排版設定的()裡可以加數字嗎?

根據題目規格，**只有中央顯示點數的 JLabel 有明確規定必須設定「字體 60pt」**。上方文字與按鈕並未硬性規定，若不額外設定，程式會直接使用 Java Swing 的預設字體。 PNG

不過在實務開發上，預設字體通常偏小。為了讓介面更好看，您可以隨時為它們補上字體設定：

```java
// 設定上方標籤字體（例如：微軟正黑體，一般粗細，16pt）
statsLabel.setFont(new Font("微軟正黑體", Font.PLAIN, 16));

// 設定按鈕字體（例如：微軟正黑體，粗體，20pt）
rollButton.setFont(new Font("微軟正黑體", Font.BOLD, 20));
```

關於排版設定，**括號裡絕對可以加數字，這些數字通常是用來設定「元件之間的間距 (Gap)」**。

以我們剛才使用的 `BorderLayout` 為例，預設的 `new BorderLayout()` 元件之間是緊緊貼在一起的（間距為 0）。如果您在括號內加入兩個數字，分別代表**水平間距 (hgap)** 與 **垂直間距 (vgap)**：

```java
// 設定 BorderLayout，並讓元件之間有 水平 10 像素、垂直 20 像素 的空隙
frame.setLayout(new BorderLayout(10, 20));
```

加入間距後，上方的統計文字、中間的骰子、以及下方的按鈕之間，就會產生呼吸空間，不會全部擠在一起。

如果您未來用到另一種常見的排版 `GridLayout`（網格排版），括號裡的數字更是必要且常見的：

```java
// GridLayout(列數, 行數, 水平間距, 垂直間距)
// 例如：設定 3 列 1 行，並且上下間距為 15 像素
panel.setLayout(new GridLayout(3, 1, 0, 15));
```

有待加強

後面會用到ActionEvent類別，是不是要在前面加上import java.awt.event.*;

是的，完全正確！

因為後面在實作按鈕點擊功能時，必定會用到 `ActionEvent` 類別以及 `ActionListener` 介面，而這兩者都隸屬於 `java.awt.event` 套件。

您可以在程式碼最上方加入這行萬用字元（`*`）匯入，這樣就能一次載入該套件下的所有事件類別：



Java

```
import java.awt.event.*;
```

或者，如果您習慣將每個用到的類別標示清楚（這種精確的寫法在實務上也很常見），也可以分開單獨匯入：

Java

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
```

這兩種寫法都可以讓您的程式順利辨識並執行後續的按鈕事件語法。

HW2-0918- Reflection

AI 剛開始給的程式碼 frame.setVisible(true);放在視窗建立完，但放在那裡會造成後面要增加的元件可能會顯示不出來，所以我把它改放在建立好所有元件後的那一行。第二個問題是少了 import java.awt.event.*;，因為後面實作時會用到這個類別，所以必須加在最前面。